

```

#include <mpi.h>
#include <stdio.h>

const int N = 1000000;

// f(x)
float f(float x) { return x * 2 / 30; }

// Calculo de integral via método do trapézio
float trap_integral_local(float a, float b, int local_n, float h) {
    float acc = (f(a) + f(b)) / 2.0f;
    for (int i = 1; i < local_n; i++) {
        float x = a + i * h;
        acc += f(x);
    }
    return acc * h;
}

int main(int argc, char **argv) {
    int world_rank, world_size;

    // Inicializa MPI
    MPI_Init(&argc, &argv);
    // Pega o rank atual
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
    // Tamanho do mundo
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    // Tem 1 nó apenas. Impossível continuar.
    if (world_size < 2) {
        if (world_rank == 0) {
            printf("Precisa de pelo menos 2 processos pra rodar esse programa.");
        }
        MPI_Finalize();
        return 0;
    }

    float global_a = 0.0f;
    float global_b = 10.0f;
    float h = (global_b - global_a) / N;

    // Nó "master", apenas espera os nós workers enviarem suas partes
    if (world_rank == 0) {
        float total_res = 0.0f;
        float received_val;

        printf("Master esperando %d resultados...\n", world_size - 1);

        // Pega cada parte e soma no resultado final.
        for (int i = 1; i < world_size; i++) {
            MPI_Recv(&received_val, 1, MPI_FLOAT, i, 0, MPI_COMM_WORLD,
                    MPI_STATUS_IGNORE);
            total_res += received_val;
        }

        printf("Integral final: %f\n", total_res);
    }
}

```

```

} else {
    int num_workers = world_size - 1;
    int local_n = N / num_workers;

    // Pega o intervalo do nó atual
    float local_a = global_a + (world_rank - 1) * local_n * h;
    float local_b = local_a + local_n * h;

    // roda o o trapezio naquele intervalo
    float local_res = trap_integral_local(local_a, local_b, local_n, h);

    printf("Resultado parcial do nó %d: %f\n", world_rank, local_res);

    // Envia mensagem pro master
    MPI_Send(&local_res, 1, MPI_FLOAT, 0, 0, MPI_COMM_WORLD);
}

MPI_Finalize();
return 0;
}

// > mpicc -Wall und2_atv2.c
// mpirun --oversubscribe -n 4 ./a.out
// Master esperando 3 resultados...
// Resultado parcial do nó 1: 0.370398
// Resultado parcial do nó 3: 1.851682
// Resultado parcial do nó 2: 1.111148
// Integral final: 3.333228

```